

Assignments

T3 – Task Three

Notice

The information in this document is subject to change without notice, and should not be construed as a commitment by HENSOLDT Cyber.

HENSOLDT Cyber provides no warranty, express or implied, including warranties of merchantability and fitness for a particular purpose for the information contained in this document, and assumes no responsibility for any errors that may appear in this document. In no event shall HENSOLDT Cyber or any of its suppliers be liable for direct, indirect, special, incidental or consequential damages of any nature or kind arising from the use of this document, or from the use of any hardware or software described in this document, even if HENSOLDT Cyber or its suppliers have been advised of the possibility of such damages.

This document and parts hereof must not be reproduced or copied without written permission from HENSOLDT Cyber and the contents hereof must not be imparted to a third party nor used for any unauthorized purpose. All rights to registrations and trademarks reside with their respective owners.

Copyright © 2021 HENSOLDT Cyber GmbH – All rights reserved.

Trademarks

TRENTOS® is a trademark of HENSOLDT Cyber GmbH.

sel4® is a trademark of LF Projects, LLC.

Table of Contents

Table of Contents	3
1 Learning Goals	6
2 General Information	6
3 TestApp as network server.....	7
3.1 What to do	7
3.2 Network Driver	7
3.3 Ticker.....	11
3.4 Network Stack	15
3.5 TestApp	24
4 TestApp as network client	34
4.1 What to do	34
4.2 Hints	35
4.3 The netcat webserver	35
4.4 Run the demo	36

Learning Goals

The main task of this self-study assignment is to extend the existing setup by networking functionality - first in server, then in client mode. The server mode will be demonstrated step by step, in order to get a feeling for the TRENTOS-M internal networking workflow. The client mode will be the task for the self-study. In order to enable network support, we want to make use of the RPi internal network interface. We will utilize a native Ethernet port on the PC or alternatively the USB-to-Ethernet adapter and the network cable which you can find in the TRENTOS Development Kit. More information regarding the RPi network interface and the respective driver can be found in the TRENTOS-M Handbook, chapter "TRENTOS-M NIC_RPi". The figure below describes the setup we want to create:

Learning Goals

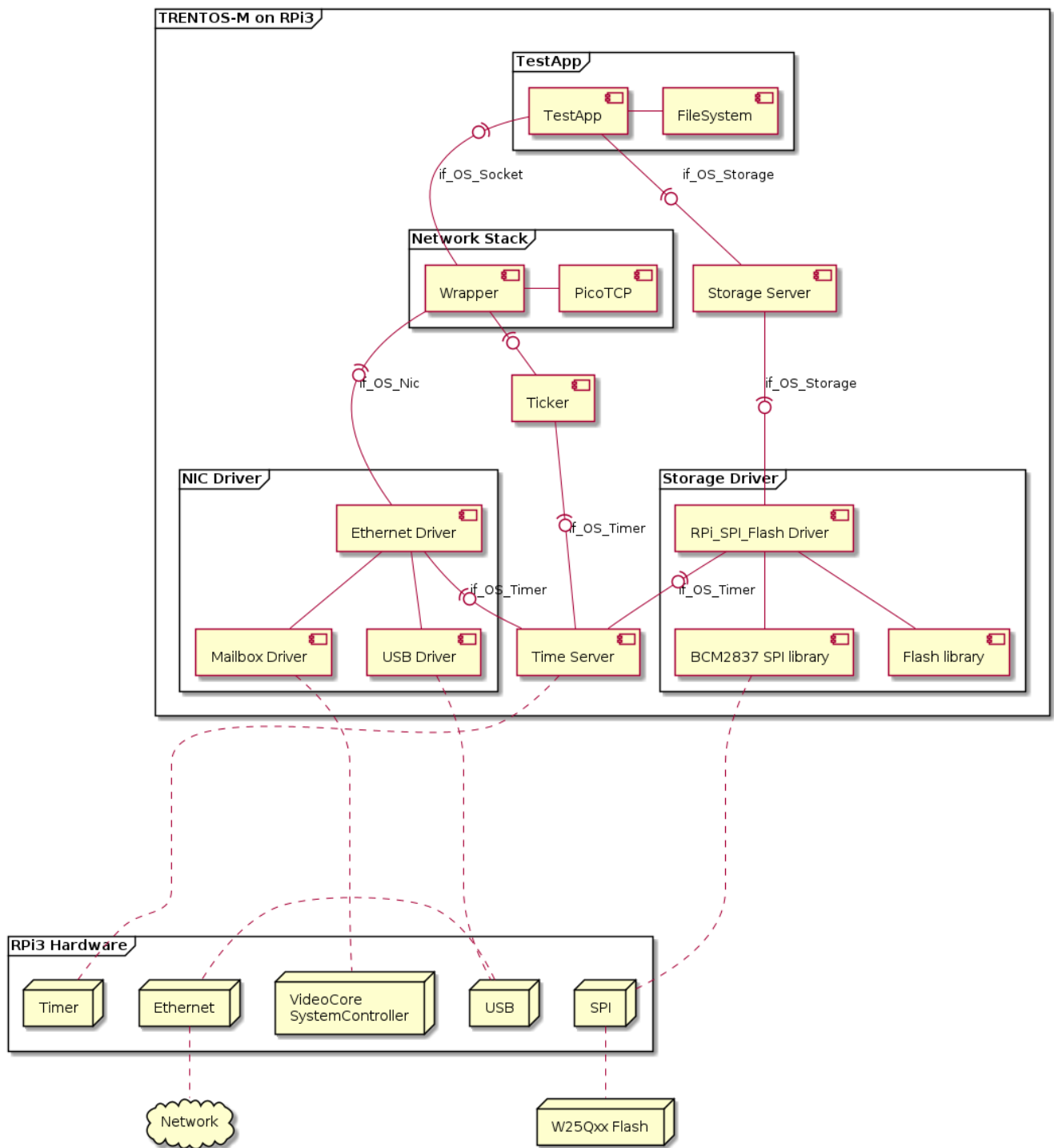


Figure 1: RPi_SPI_Flash Driver - Architecture

1 Learning Goals

As a developer you should understand

- the basic concept of a NIC driver in TRENTOS-M
- the usage of the TRENTOS-M standard interface `if_OS_Nic`
- the utilization of a network stack on top of a NIC driver (both in server and client mode)
- basic network communication on top of the network stack by using `if_OS_Socket`

2 General Information

Please wire the RPi 3 B+ and the PC via the network cable (and in case required use the USB-to-Ethernet adapter). As we will use a fix configuration for our demo, we have to assign the PC's network interface (or alternatively the USB-to-Ethernet adapter) a static IP address on the Linux host, which will be in our case the 10.0.0.10. The RPi will receive the static IP address 10.0.0.11, which will be assigned later on within the demo application (see the instructions in the sections below).

3 TestApp as network server

At first, we want to enhance our `TestApp` in order to allow the RPi to act like a network server that can receive data from the outside. On the opposite side, the PC will act as a client that can send data to the RPi. In our basic setup, we design the network server as an echo server and therefore want to

- utilize the Linux tool `netcat` in order to send the simple String "hello" from the PC to the RPi
- receive the String within our `TestApp`
- send the String back to the Linux PC with `netcat` visualizing the "echo" and
- store the String in a file on the SPI NOR Flash with the help of our existing file system setup

As all the basic file system and storage related parts are already there, we just have to focus on the networking aspects and some small modifications to the `TestApp` in order to handle the received String accordingly.

Hint: The parts of the C code or CAMkES descriptions that remain unchanged from the previous self-study setup will be abbreviated by three dots (...).

3.1 What to do

As can be seen from the figure above, the networking setup requires three additional components:

- a network driver
- a ticker
- a network stack

As a network driver we can just re-use the TRENTOS-M component `NIC_RPi`. Both the `Ticker` and `NetworkStack` we have to provide on our own. Fortunately, we can again rely on the TRENTOS-M API. In addition, we also have to adapt our `TestApp` for being able to use the `if_OS_Socket` interface for network communication.

3.2 Network Driver

More detailed information regarding the RPi network driver (`NIC_RPi`) internals can be found in the TRENTOS-M Handbook, chapter "TRENTOS-M `NIC_RPi`". As can be seen in the figure above, the RPi network driver has to interact with a few hardware components in order to provide sufficient network functionality. The component can be seen as a black box which abstracts the driver complexity from the user. Via the TRENTOS-M standard interface `if_OS_Nic`, the network driver can be connected to a network stack running on top.

We have to follow again the typical steps when re-using a component. At first, we have to modify our existing `CMakeLists.txt` file, in order to declare the `NIC_RPi` component.

TestApp as network server

```
...
NIC_RPi_DeclareCAMEComponent (
    NIC_RPi
)
...
```

Code Block 1: CMakeLists.txt

Afterwards, we have to declare the `NIC_RPi` component in our existing `DemoTrainingCourse.camkes` file. As we can see from the snippet below, `NIC_RPi` actually consists of three different components which reflect the RPi specific way of networking. More details on the RPi internals can be found in the TRENTOS-M Handbook, chapter "TRENTOS-M NIC_RPi". By now, we just need to know that we have to declare all of them.

```
...
//-----
// Network
//-----

#include "NIC_RPi/NIC_RPi.camkes"
NIC_RPi_COMPONENT_DEFINE(NIC_RPi, NIC_DRIVER_RINGBUFFER_SIZE)
NIC_RPi_Mailbox_COMPONENT_DEFINE(NIC_RPi_Mailbox)
NIC_RPi_USB_COMPONENT_DEFINE(NIC_RPi_USB)
...
```

Code Block 2: DemoTrainingCourse.camkes

Within the `composition` part of our application, we then also have to instantiate and connect the different components that make up `NIC_RPi` and connect the network driver instance as a client to the `TimeServer`.

TestApp as network server

```
composition {  
  ...  
  //-----  
  // Ethernet Driver  
  //-----  
  component NIC_RPi          nic;  
  component NIC_RPi_Mailbox  nic_mailbox;  
  component NIC_RPi_USB      nic_usb;  
  
  NIC_RPi_INSTANCE_CONNECT(  
    nic,  
    nic_mailbox,  
    nic_usb  
  )  
  
  ...  
  TimeServer_INSTANCE_CONNECT_CLIENTS(  
    timeServer,  
    ...  
    nic.timeServer_rpc, nic.timeServer_notify  
  )  
  
  ...  
}
```

Code Block 3: DemoTrainingCourse.camkes

Finally, we have to configure the driver by specifying relevant parameters within the `configuration` section.

TestApp as network server

```
configuration {  
    ...  
    TimeServer_CLIENT_ASSIGN_BADGES(  
        ...  
        nic.timeServer_rpc  
    )  
  
    NIC_RPi_Mailbox_INSTANCE_CONFIGURE_SELF(  
        nic_mailbox  
    )  
    NIC_RPi_USB_INSTANCE_CONFIGURE_SELF(  
        nic_usb  
    )  
    NIC_RPi_INSTANCE_CONFIGURE(  
        nic,  
        4*40960  
    )  
    ...  
}
```

Code Block 4: DemoTrainingCourse.camkes

As we can see above, `NIC_RPi` requires the parameter `NIC_DRIVER_RINGBUFFER_SIZE` during declaration. We have to provide a corresponding value for that within our existing global configuration, the `system_config.h`.

```
...  
//-----  
// NIC driver  
//-----  
#define NIC_DRIVER_RINGBUFFER_NUMBER_ELEMENTS 16  
#define NIC_DRIVER_RINGBUFFER_SIZE \\\n    (NIC_DRIVER_RINGBUFFER_NUMBER_ELEMENTS * 4096)  
...  

```

Code Block 5: system_config.h

3.3 Ticker

The Ticker provides the required tick to our network stack. Internally it re-uses the TRENTOS-M `TimeServer` component, basically in the same way as we have already seen when using the SPI NOR flash driver. For the `Ticker`, we basically follow the steps for creating a new component, which were already described in detail within the basic setup.

First of all, within the folder `components`, we will create a new folder `Ticker`. Inside this folder, we will place the `Ticker` component description in form of the `Ticker.camkes` file.

```
/*
 * Ticker
 *
 * Copyright (C) 2020, Hensoldt Cyber GmbH
 */

import <if_OS_Timer.camkes>;

component Ticker {
    control;

    uses      if_OS_Timer      timeServer_rpc;
    consumes  TimerReady       timeServer_notify;

    emits     EventDataAvailable nwStack_event_tick;
}
```

Code Block 6: Ticker.camkes

The `Ticker` component shall use the `TimeServer`. In order to allow for communication between the component and the underlying `TimeServer`, the component has to be equipped with an RPC interface and must be enabled to deal with events. As an interface between the `Ticker` and the `TimeServer` the TRENTOS-M standard interface `if_OS_Timer` (see `import`) is used. The `Ticker` component is therefore able to use the functions which are defined by the `if_OS_Timer` interface in the file `if_OS_Timer.camkes`. The respective function logic is implemented by the `TimeServer` component. The `Ticker` can additionally receive (keyword `consumes`) events from the `TimeServer` and will in contrary send (keyword `emits`) events (here: the required tick) to the `NetworkStack`.

We then have to modify our existing `CMakeLists.txt` file, in order to declare the `Ticker` component.

TestApp as network server

```
...
DeclareCAMkESComponent (
    Ticker
    SOURCES
        components/Ticker/Ticker.c
    C_FLAGS
        -Wall
        -Werror
    LIBS
        lib_debug
        os_core_api
)
...
```

Code Block 7: CMakeLists.txt

We now have to declare the `Ticker` component in our existing existing `DemoTrainingCourse.camkes` file.

```
...
//-----
// Ticker
//-----

import "components/Ticker/Ticker.camkes";
...
```

Code Block 8: DemoTrainingCourse.camkes

Within the `composition` part of our application, we then have to instantiate the `Ticker` component and connect it as an additional client to our already declared `TimeServer` from the previous SPI NOR Flash setup.

TestApp as network server

```
composition {
...
    //-----
    // Ticker
    // It's a very pragmatic approach to create a 1 second tick for each
    // network stack. Actually, we don't need this, because the TimeServer
    // should provide two timers per client, one for timeouts and one for
    // the periodic tick.
    //-----
    component Ticker ticker;

...
...
...

    TimeServer_INSTANCE_CONNECT_CLIENTS (
        timeServer,
        ...
        ticker.timeServer_rpc, ticker.timeServer_notify
    )

...
}
```

Code Block 9: DemoTrainingCourse.camkes

We also have to configure the `Ticker` by specifying relevant parameters within the `configuration` section.

```
configuration {
...
    TimeServer_CLIENT_ASSIGN_BADGES (
        ...
        ticker.timeServer_rpc
    )

...
}
```

Code Block 10: DemoTrainingCourse.camkes

In a next step, within the `Ticker` folder we create a new file called `Ticker.c`, which shall contain the actual `Ticker` component logic.

TestApp as network server

```
/*
 * Ticker
 *
 * Copyright (C) 2020, Hensoldt Cyber GmbH
 *
 */

#include "lib_debug/Debug.h"

#include <camkes.h>

//-----
int run(void)
{
    Debug_LOG_INFO("ticker running");

    // set up a tick every second
    int ret = timeServer_rpc_periodic(0, NS_IN_S);
    if (0 != ret)
    {
        Debug_LOG_ERROR("timeServer_rpc_periodic() failed, code %d", ret);
        return -1;
    }

    for(;;)
    {
        timeServer_notify_wait();

        // send tick to network stack
        nwStack_event_tick_emit();
    }
}
```

Code Block 11: Ticker.c

The `Ticker` waits for the according event (`timeServer_notify_wait()`) from the `TimeServer` and then emits an event to the `NetworkStack` (`nwStack_event_tick_emit()`).

3.4 Network Stack

The network stack provides IP functionality for the system, allowing TCP and UDP communication to be performed. More detailed information as well as a reference setup can be found in the TRENTOS-M Handbook, chapter "TRENTOS-M Network API". As for the `Ticker` before, within the folder `components`, we will create a new folder `NetworkStack`. Inside this folder, we will place the `NetworkStack` component description in form of the `NetworkStack.camkes` file.

```
/*
 * Network Stack
 *
 * Copyright (C) 2020, Hensoldt Cyber GmbH
 */

import <if_OS_Nic.camkes>;
import <if_OS_Socket.camkes>;
import <if_OS_Timer.camkes>;

#include "include/util/loop_defines.h"
#include "system_config.h"

component NetworkStack {

    control;

    emits      EventDataAvailable      e_tick_or_data;
    consumes   EventDataAvailable      c_tick_or_data; // event also from NIC
and Timer

    has        mutex                    allocatorMutex;
    has        mutex                    nwstackMutex;

    has        mutex                    socketControlBlockMutex;
    has        mutex                    stackThreadSafeMutex;

#define LOOP_COUNT OS_NETWORK_MAXIMUM_SOCKET_NO
#define LOOP_ELEMENT \
    GEN_ID(emits NwStack_WrEv e_write); \
    GEN_ID(consumes NwStack_WrEv c_write); \
    GEN_ID(emits NwStack_RdEv e_read); \
    GEN_ID(consumes NwStack_RdEv c_read); \
    GEN_ID(emits NwStack_ConnEv e_conn); \
    GEN_ID(consumes NwStack_ConnEv c_conn);
#include "include/util/loop.h"
```

TestApp as network server

```
//-----
// interface TimeServer
uses      if_OS_Timer      timeServer_rpc;
consumes   TimerReady      timeServer_notify;

//-----
// interface Ethernet NIC driver
uses      if_OS_Nic      nic_driver;
dataport   Buf(NIC_DRIVER_RINGBUFFER_SIZE) nic_port_from; // NIC -> stack
dataport   Buf      nic_port_to; // stack -> NIC

//-----
// interface Application
provides if_OS_Socket network_stack_rpc; // name hard-coded in network client lib
emits EventServiceReady nwStack_event_ready;
#define LOOP_COUNT OS_NETWORK_MAXIMUM_SOCKET_NO
#define LOOP_ELEMENT \
    GEN_ID(dataport Buf nwStack_port);
#include "include/util/loop.h"
}
```

Code Block 12: NetworkStack.camkes

The `NetworkStack` component shall be able to use functionality provided by the `TimeServer`, the `Ticker` and the NIC driver (here: `NIC_RPi`). In order to allow for communication between these different components, the `NetworkStack` has to be equipped with the required RPC interfaces and must be enabled to deal with events. The most important building blocks will be described in the following:

- As an interface between the `NetworkStack` and the `TimeServer` the already known TRENTOS-M standard interface `if_OS_Timer` is used.
- The `Ticker` communicates via an event (`c_tick_or_data`) with the `NetworkStack`, in order to provide the required timer tick.
- In order to enable networking, the `NetworkStack` communicates via the TRENTOS-M standard interface `if_OS_Nic` with the `NIC_RPi` network driver. For exchanging data in a bidirectional way, two dataports (`nic_port_from` and `nic_port_to`) are used.
- Our `TestApp` can communicate with the `NetworkStack` via the TRENTOS-M standard interface `if_OS_Socket`, which must be named `network_stack_rpc` when using functionality provided by the network client library, as this identifier is currently hard-coded there. It gets notified about new data available via a respective event (`nwStack_event_ready`) and uses a dataport (`nwStack_port`) for data exchange.

For supporting the potential use case that multiple socket connections shall be utilized, we prepare our network stack accordingly by using specific macros (here: `LOOP_COUNT`, `LOOP_ELEMENT` and `GEN_ID`). The exact amount of sockets depends on the value of `OS_NETWORK_MAXIMUM_SOCKET_NO`,

TestApp as network server

which has to be specified within our existing global configuration, the `system_config.h`. Besides that, we also define respective values for the IP address configuration. These values are provided to the network stack implementation via the `CMakeLists.txt` and can therefore be used within the actual C code.

```
...
//-----
// Network Stack
//-----

#ifndef OS_NETWORK_MAXIMUM_SOCKET_NO
#define OS_NETWORK_MAXIMUM_SOCKET_NO 16
#endif

#define ETH_ADDR                "10.0.0.11"
#define ETH_GATEWAY_ADDR        "10.0.0.1"
#define ETH_SUBNET_MASK         "255.255.255.0"
...
```

Code Block 13: system_config.h

For being able to use the above mentioned macros, we have to create a new folder `include` within the root directory, which shall contain a new folder `util`. Within this folder we create two new files, which contain the required definitions:

- `loop_defines.h`

```
/* Copyright (C) 2020, Hensoldt Cyber GmbH */
#define CONCAT_IMPL(X, Y) X##Y

#define MACRO_CONCAT(X, Y) CONCAT_IMPL(X, Y)

#define GEN_ID(X) MACRO_CONCAT(X, __INCLUDE_LEVEL__)

#define GEN_EMIT(X) MACRO_CONCAT(MACRO_CONCAT(X, __INCLUDE_LEVEL__), _emit)

#define GEN_WAIT(X) MACRO_CONCAT(MACRO_CONCAT(X, __INCLUDE_LEVEL__), _wait)

#define GEN_NAME(X, Y) MACRO_CONCAT(MACRO_CONCAT(X, Y), __INCLUDE_LEVEL__)
```

Code Block 14: loop_defines.h

- `loop.h`

TestApp as network server

```
/* Copyright (C) 2020, Hensoldt Cyber GmbH */
#ifndef LOOP_COUNT
#error "LOOP_COUNT not set"
#endif

#if __INCLUDE_LEVEL__ <= LOOP_COUNT
LOOP_ELEMENT
#include "loop.h"
#else
#undef LOOP_COUNT
#undef LOOP_ELEMENT
#endif
```

Code Block 15: loop.h

We then have to modify our existing `CMakeLists.txt` file, in order to declare the `NetworkStack` component.

```
...
DeclareCamkesComponent (
    NetworkStack
    INCLUDES
        include/util
    SOURCES
        components/NetworkStack/NetworkStack.c
    C_FLAGS
        -Wall -Werror
        -DCFG_ETH_ADDR=ETH_ADDR
        -DCFG_ETH_GATEWAY_ADDR=ETH_GATEWAY_ADDR
        -DCFG_ETH_SUBNET_MASK=ETH_SUBNET_MASK
    LIBS
        system_config
        os_core_api
        lib_debug
        os_network_lib
        TimeServer_client
)
...
```

Code Block 16: CMakeLists.txt

Now we declare the `NetworkStack` component in our existing `DemoTrainingCourse.camkes` file.

TestApp as network server

```
...
//-----
// Network Stack
//-----

import "components/NetworkStack/NetworkStack.camkes";
#include "include/util/loop_defines.h"
...
```

Code Block 17: DemoTrainingCourse.camkes

Within the `composition` part of our application, we then have to instantiate the `NetworkStack` component and connect its internal interfaces. In addition, we also connect it as an additional client to our already declared `TimeServer`. We then have to connect the `Ticker` event (`nwStack_event_tick`) to the corresponding `NetworkStack` event (`c_tick_or_data`). As a last step, we also connect the network stack to the network driver (`NIC_RPi`) with the help of the `NIC_RPi_INSTANCE_CONNECT_CLIENT` macro.

```
composition {
...
    //-----
    // Network Stack
    //-----

    component NetworkStack          nwStack;

#define COMPONENT nwStack
#define LOOP_COUNT OS_NETWORK_MAXIMUM_SOCKET_NO
#define LOOP_ELEMENT \
    connection seL4Notification GEN_NAME(COMPONENT, _a) (from
COMPONENT.GEN_ID(e_write), to COMPONENT.GEN_ID(c_write)); \
    connection seL4Notification GEN_NAME(COMPONENT, _b) (from
COMPONENT.GEN_ID(e_read), to COMPONENT.GEN_ID(c_read)); \
    connection seL4Notification GEN_NAME(COMPONENT, _c) (from
COMPONENT.GEN_ID(e_conn), to COMPONENT.GEN_ID(c_conn));
#include "include/util/loop.h"
#undef COMPONENT
...
...
...
    TimeServer_INSTANCE_CONNECT_CLIENTS (
        timeServer,
        ...
        nwStack.timeServer_rpc, nwStack.timeServer_notify
    )
}
```

TestApp as network server

```
...
...
...
        connection sel4NotificationNative nwStack_tick (from ticker.nwStack_event_tick,
to nwStack.c_tick_or_data);
...
...
...
        NIC_RPi_INSTANCE_CONNECT_CLIENT(
            nic,
            nwStack.nic_driver, nwStack.nic_port_to, nwStack.nic_port_from,
nwStack.c_tick_or_data
        )
...

```

Code Block 18: DemoTrainingCourse.camkes

We also have to configure the `NetworkStack` by specifying relevant parameters within the `configuration` section.

```
configuration {
...
        TimeServer_CLIENT_ASSIGN_BADGES(
            ...
            nwStack.timeServer_rpc
        )
...

```

Code Block 19: DemoTrainingCourse.camkes

Finally, for making use of the TRENTOS-M Network API, we need to create a new file called `NetworkStack.c` within `/components/NetworkStack` which shall contain the actual `NetworkStack` component logic.

```
/*
 * OS Network Stack CAmkES wrapper
 *
 * Copyright (C) 2019, Hensoldt Cyber GmbH
 *
 */

#include "system_config.h"

```

TestApp as network server

```
#include "lib_debug/Debug.h"
#include "OS_Error.h"
#include "OS_NetworkStack.h"
#include "TimeServer.h"
#include <camkes.h>
#include "OS_Dataport.h"
#include "loop_defines.h"

static const if_OS_Timer_t timer =
    IF_OS_TIMER_ASSIGN(
        timeServer_rpc,
        timeServer_notify);

static const OS_NetworkStack_AddressConfig_t config =
{
    .dev_addr      = CFG_ETH_ADDR,
    .gateway_addr  = CFG_ETH_GATEWAY_ADDR,
    .subnet_mask   = CFG_ETH_SUBNET_MASK
};

//-----
// network stack's PicoTCP OS adaption layer calls this.
uint64_t
Timer_getTimeMs(void)
{
    OS_Error_t err;
    uint64_t ms;

    if ((err = TimeServer_getTime(&timer, TimeServer_PRECISION_MSEC,
                                  &ms)) != OS_SUCCESS)
    {
        Debug_LOG_ERROR("TimeServer_getTime() failed with %d", err);
        ms = 0;
    }

    return ms;
}

//-----
int run(void)
{
    Debug_LOG_INFO("[NwStack '%s'] starting", get_instance_name());

    #define LOOP_ELEMENT \
        { \
            .notify_write    = GEN_EMIT(e_write), \
```

TestApp as network server

```

        .wait_write      = GEN_WAIT(c_write), \
        .notify_read     = GEN_EMIT(e_read), \
        .wait_read       = GEN_WAIT(c_read), \
        .notify_connection = GEN_EMIT(e_conn), \
        .wait_connection  = GEN_WAIT(c_conn), \
        .buf              = OS_DATAPORT_ASSIGN(GEN_ID(nwStack_port)), \
        .accepted_handle  = -1, \
    },

static OS_NetworkStack_SocketResources_t socks[OS_NETWORK_MAXIMUM_SOCKET_NO] = {
    #define LOOP_COUNT OS_NETWORK_MAXIMUM_SOCKET_NO
    #include "loop.h" // places LOOP_ELEMENT here for LOOP_COUNT times
};

static const OS_NetworkStack_CamkesConfig_t camkes_config =
{
    .notify_init_done      = nwStack_event_ready_emit,
    .wait_loop_event       = c_tick_or_data_wait,

    .internal =
    {
        .notify_loop       = e_tick_or_data_emit,

        .allocator_lock    = allocatorMutex_lock,
        .allocator_unlock  = allocatorMutex_unlock,

        .nwStack_lock      = nwstackMutex_lock,
        .nwStack_unlock    = nwstackMutex_unlock,

        .socketCB_lock     = socketControlBlockMutex_lock,
        .socketCB_unlock   = socketControlBlockMutex_unlock,

        .stackTS_lock      = stackThreadSafeMutex_lock,
        .stackTS_unlock    = stackThreadSafeMutex_unlock,

        .number_of_sockets = OS_NETWORK_MAXIMUM_SOCKET_NO,
        .sockets           = socks,
    },

    .drv_nic =
    {
        .from =
        {
            .io = (void**)(&(nic_port_from)),
            .size = NIC_DRIVER_RINGBUFFER_NUMBER_ELEMENTS
        },
    },
};

```

TestApp as network server

```
.to = OS_DATAPORT_ASSIGN(nic_port_to),

.rpc =
{
    .dev_read      = nic_driver_rx_data,
    .dev_write     = nic_driver_tx_data,
    .get_mac       = nic_driver_get_mac_address,
}

},

.app =
{
    .notify_init_done = nwStack_event_ready_emit,
}

};

// The Ticker component sends us a tick every second. Currently there is
// no dedicated interface to enable and disable the tick. because we don't
// need this. OS_NetworkStack_run() is not supposed to return.

OS_Error_t ret = OS_NetworkStack_run(&camkes_config, &config);
if (ret != OS_SUCCESS)
{
    Debug_LOG_FATAL("[NwStack '%s'] OS_NetworkStack_run() failed, error %d",
                    get_instance_name(), ret);

    return -1;
}

// actually, OS_NetworkStack_run() is not supposed to return with
// OS_SUCCESS. We have to assume this is a graceful shutdown for some
// reason
Debug_LOG_WARNING("[NwStack '%s'] graceful termination",
                  get_instance_name());

return 0;
}
```

Code Block 20: NetworkStack.c

TestApp as network server

According to the TRENTOS-M Handbook, chapter "TRENTOS-M Network API", the following aspects are mandatory:

- the provision of a struct `camkes_config`, which contains references to the actual RPC and event related functions generated by CAMkES based on the CAMkES descriptions provided by `NetworkStack.camkes`, e.g.
 - `nwStack_event_ready_emit` as the concrete function name to be used, which is made up by the concatenation of the event name `nwStack_event_ready` and the `emit` function provided by CAMkES to send an event
 - `nic_driver_rx_data` as the concrete function name to be used, which is made up by the concatenation of the `if_OS_Nic` RPC interface named `nic_driver` and the function name `rx_data` specified in `/sdk/libs/os_core_api/camkes/if_OS_Nic.camkes`
- the provision of a struct `config`, which contains the IP address related information provided by the `system_config.h`
- a call of `OS_NetworkStack_run()` which uses the information provided by both structs in order to start the actual network stack

3.5 TestApp

We have to modify our existing `TestApp` component description in form of the `TestApp.camkes` file and add the required support for networking. As we want to communicate with the network stack, we have to use the TRENTOS-M `if_OS_Socket` interface. The building blocks added here reflect the building blocks already described in the `NetworkStack.camkes` file above.

```
...
import <if_OS_Socket.camkes>;
...
#include "include/util/loop_defines.h"
...
component TestApp {
...
...
...
    // Networking
    uses      if_OS_Socket      network_stack_rpc;
    consumes  EventServiceReady event_network_stack_init_done;

#define LOOP_COUNT OS_NETWORK_MAXIMUM_SOCKET_NO
#define LOOP_ELEMENT \
    GEN_ID(dataport Buf NwAppDataPort);
#include "include/util/loop.h"
...
}
```

Code Block 21: TestApp.camkes

TestApp as network server

We then have to modify our existing `CMakeLists.txt` file, in order to extend our `TestApp` component for using the TRENTOS-M Network API by adding `os_network_api` to LIBS.

```
...
DeclareCamkESComponent(
    TestApp
    INCLUDES
        include/util
    SOURCES
        components/TestApp/src/TestApp.c
    C_FLAGS
        -Wall
        -Werror
    LIBS
        system_config
        lib_debug
        os_core_api
        os_filesystem
        os_network_api
)
...
```

Code Block 22: CMakeLists.txt

Within the `composition` part of our existing `DemoTrainingCourse.camkes` file, we then have to connect the `NetworkStack` component and the `TestApp` component.

```
composition {
    ...
    ...
    ...
    connection seL4Notification      NwAppInitDone      (from
nwStack.nwStack_event_ready, to testApp.event_network_stack_init_done);
    connection seL4RPCall            testApp_nwStack      (from
testApp.network_stack_rpc, to nwStack.network_stack_rpc);

#define LOOP_COUNT OS_NETWORK_MAXIMUM_SOCKET_NO
#define LOOP_ELEMENT \
    connection seL4SharedData GEN_NAME(testApp, nwStack) (from
testApp.GEN_ID(NwAppDataPort), to nwStack.GEN_ID(nwStack_port));
#include "include/util/loop.h"
    ...
}
```

Code Block 23: DemoTrainingCourse.camkes

TestApp as network server

As a last step we have to adapt our `TestApp.c` file, in order to reflect all the changes made in the CAMkES descriptions and for providing the respective network server logic.

```
...
#include "OS_Error.h"
#include "OS_Network.h"
#include "OS_NetworkStackClient.h"
#include "loop_defines.h"
...
static char fileData[5];
...
...
...
//-----
// Network
//-----

extern OS_Error_t OS_NetworkAPP_RT(OS_Network_Context_t ctx);

void init_client_api()
{
    static OS_NetworkStackClient_SocketDataports_t config;

    config.number_of_sockets = OS_NETWORK_MAXIMUM_SOCKET_NO;
    static OS_Dataport_t dataports[OS_NETWORK_MAXIMUM_SOCKET_NO] = {0};

    int i = 0;

#define LOOP_COUNT OS_NETWORK_MAXIMUM_SOCKET_NO
#define LOOP_ELEMENT \
    GEN_ID(OS_Dataport_t t) = OS_DATAPORT_ASSIGN(GEN_ID(NwAppDataPort)); \
    dataports[i] = GEN_ID(t); \
    i++;
#include "loop.h"

    config.dataport = dataports;
    OS_NetworkStackClient_init(&config);
}

//-----
int run()
{
    //-----
    // Network Test
    //-----
}
```

TestApp as network server

```
init_client_api();

Debug_LOG_INFO("Starting test_app_server...");

char buffer[4096];
OS_NetworkAPP_RT(NULL);    /* Must be actually called by OS Runtime */

OS_NetworkServer_Socket_t  srv_socket =
{
    .domain = OS_AF_INET,
    .type    = OS SOCK_STREAM,
    .listen_port = 5555,
    .backlog = 1,
};

/* Gets filled when accept is called */
OS_NetworkSocket_Handle_t  os_socket_handle ;
/* Gets filled when server socket create is called */
OS_NetworkServer_Handle_t  os_nw_server_handle ;

OS_Error_t err = OS_NetworkServerSocket_create(NULL, &srv_socket,
                                                &os_nw_server_handle);

if (err != OS_SUCCESS)
{
    Debug_LOG_ERROR("server_socket_create() failed, code %d", err);
    return -1;
}

Debug_LOG_INFO("launching echo server");

for (;;)
{
    err = OS_NetworkServerSocket_accept(os_nw_server_handle, &os_socket_handle);
    if (err != OS_SUCCESS)
    {
        Debug_LOG_ERROR("socket_accept() failed, error %d", err);
        return -1;
    }

    /*
    As of now the nw stack behavior is as below:
    Keep reading data until you receive one of the return values:
    a. err = OS_ERROR_CONNECTION_CLOSED and length = 0 indicating end of data
    read
    */
}
```

TestApp as network server

```
        and connection close
    b. err = OS_ERROR_GENERIC due to error in read
    c. err = OS_SUCCESS and length = 0 indicating no data to read but there is
still
        connection
    d. err = OS_SUCCESS and length >0 , valid data

Take appropriate actions based on the return value rxd.

Only a single socket is supported and no multithreading !!!
Once we accept an incoming connection, start reading data from the client and
echo back
    the data rxd.
*/
memset(buffer, 0, sizeof(buffer));

Debug_LOG_INFO("starting server read loop");
/* Keep calling read until we receive CONNECTION_CLOSED from the stack */
for (;;)
{
    Debug_LOG_DEBUG("read...");
    size_t n = 0;
    // Try to read as much as fits into the buffer
    err = OS_NetworkSocket_read(os_socket_handle, buffer, sizeof(buffer), &n);
    if (OS_SUCCESS != err)
    {
        Debug_LOG_ERROR("socket_read() failed, error %d", err);
        break;
    }

    err = OS_NetworkSocket_write(os_socket_handle, buffer, n, &n);
    if (err != OS_SUCCESS)
    {
        Debug_LOG_ERROR("socket_write() failed, error %d", err);
        break;
    }

    memcpy(fileData, buffer, sizeof(fileData));
}

// -----
// Storage Test
// -----

// Work on file system 1 (residing on partition 1)
test_OS_FileSystem(&spiffsCfg_1);
```

TestApp as network server

```
// Work on file system 2 (residing on partition 2)
test_OS_FileSystem(&spiffsCfg_2);

Debug_LOG_INFO("Demo completed successfully.");

switch (err)
{
/* This means end of read as socket was closed. Exit now and close handle*/
case OS_ERROR_CONNECTION_CLOSED:
    Debug_LOG_INFO("connection closed by server");
    OS_NetworkSocket_close(os_socket_handle);
    continue;
/* Any other value is a failure in read, hence exit and close handle */
default :
    Debug_LOG_ERROR("server socket failure, error %d", err);
    OS_NetworkSocket_close(os_socket_handle);
    continue;
} //end of switch
}
return -1;
}
```

Code Block 24: TestApp.c

We basically leave all file system related configuration untouched, but we

- extract the existing `fileData` char array from our function `test_OS_FileSystem()` and transform it to a global variable that can be accessed by both the network and file system related functionality
- provide `fileData` with a size of 5 as we only want to store the "hello" String
- modify the `run()` function to call `test_OS_FileSystem()` after the network server read loop has finished (in our case by manually closing the connection from the PC side) in order to store the String on the SPI NOR Flash

We then provide the required initialization of our network stack within the function `init_client_api()`. We also modify the `run()` function to handle receiving and sending of data accordingly.

A successful run should provide you with an output similar like the one below:

TestApp as network server

```
U-Boot 2020.07-rc3-00002-g4b6f96a0a6 (May 26 2020 - 18:38:13 +0200)

DRAM:  948 MiB
RPI 3 Model B+ (0xa020d3)
MMC:   mmc@7e202000: 0, sdhci@7e300000: 1
Loading Environment from FAT... OK
In:     serial
Out:    vidconsole
Err:    vidconsole
Net:    No ethernet found.
starting USB...
Bus usb@7e980000: USB DWC2
scanning bus usb@7e980000 for devices... 4 USB Device(s) found
    scanning usb for storage devices... 0 Storage Device(s) found
Hit any key to stop autoboot:  0
8868580 bytes read in 370 ms (22.9 MiB/s)
## Starting application at 0x00ba8000 ...

ELF-loader started on CPU: ARM Ltd. Cortex-A53 r0p4
    paddr=[ba8000..13fd4c7]
No DTB passed in from boot loader.
Looking for DTB in CPIO archive...found at cb4294.
Loaded DTB from cb4294.
    paddr=[3c000..3ffff]
ELF-loading image 'kernel'
    paddr=[0..3bfff]
    vaddr=[e0000000..e003bfff]
    virt_entry=e0000000
ELF-loading image 'capdl-loader'
    paddr=[40000..7a8fff]
    vaddr=[10000..778fff]
    virt_entry=1860c
Enabling MMU and paging
Jumping to kernel-image entry point...

Bootstrapping kernel
Booting all finished, dropped to user space
main@main.c:2114 Starting CapDL Loader...
main@main.c:2116 CapDL Loader done, suspending...
spt_handle_irq@spt.c:172 handle irq called when no interrupt pending

INFO: /host/sdk/demos/demo_training_course/src/components/Ticker/Ticker.c:15: ticker
running
INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:168:
Starting test_app_server...
INFO:
```

TestApp as network server

```
/host/sdk/demos/demo_training_course/src/components/NetworkStack/NetworkStack.c:51: [Nw
DEBUG uspi: - Initializing USPi library 2.00WARNING:
/host/sdk/components/NIC_RPi/environment.c:105: Not impleme    INFO:
/host/sdk/components/RPi_SPI_Flash/RPi_SPI_Flash.c:132: BCM2837_SPI_Flash init
    INFO: /host/sdk/componeStack 'nwStack'] starting
nted!
nts/RPi_SPI_Flash/RPi_SPI_Flash.c:197: BCM2837_SPI_Flash done
    INFO: /host/sdk/components/StorageServer/server/src/storageServer_rpc.c:504: storage
medium size: 8388608 bytes
    INFO: /host/sdk/components/StorageServer/server/src/storageServer_rpc.c:522: client 1:
offset=0, size=32768
    INFO: /host/sdk/components/StorageServer/server/src/storageServer_rpc.c:522: client 2:
offset=32768, size=32768

NOTICE usbdev0-1: - Device ven424-2514, dev9-0-2 found
NOTICE usbdev0-1: - Interface int9-0-1 found
WARNING usbdev0-1: - Function is not supported
NOTICE usbdev0-1: - Interface int9-0-2 found
NOTICE usbdev: - Using device/interface int9-0-2
NOTICE usbdev0-1: - Device ven424-2514, dev9-0-2 found
NOTICE usbdev0-1: - Interface int9-0-1 found
WARNING usbdev0-1: - Function is not supported
NOTICE usbdev0-1: - Interface int9-0-2 found
NOTICE usbdev: - Using device/interface int9-0-2
NOTICE usbdev0-1: - Device ven424-7800 found
NOTICE usbdev: - Using device/interface ven424-7800
DEBUG lan7800: - MAC address is B8:27:EB:52:4A:44
DEBUG usbhub: - Port 1: Device configured
DEBUG usbhub: - Port 1: Device configured
DEBUG dwroot: - Device configured
DEBUG uspi: - USPi library successfully initialized    INFO:
/host/sdk/components/NIC_RPi/ethernetDriver.c:45: [EthDrv 'nic'] starting
    INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico_nic.c:219:
MAC: b8:27:eb:52:4a:44
    INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico_nic.c:230:
PicoTCP Device created: trentos_nic_driver
    INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_core.c:404:
signal network stack init done
    INFO: /host/sdk/libs/os_network_stack/network_core/src/network_config.c:231:
notify_app_init_done
WARNING: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico_nic.c:129:
Falling back to legacy interface.
    INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico.c:311:
[socket 0/0x29a278] created new socket
    INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico.c:397:
[socket 0/0x29a278] binding to port 5555
```

TestApp as network server

```
INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:195:
launching echo server
```

Code Block 25: RPi 3 B+ Terminal

We then open a new terminal and launch `netcat` with the following command:

```
$ nc 10.0.0.11 5555
```

Code Block 26: PC Terminal

We want to connect to the RPi which has been assigned the IP address 10.0.0.11 (see `system_config.h`) on port 5555. This is the port our `TestApp` is currently listening to, as can be seen in `TestApp.c`. After hitting <Enter>, `netcat` (with the IP address 10.0.0.10) connects to our network server, which can be seen from the `picocom` output:

```
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico.c:161:
[socket 0x29a278] connection from 10.0.0.10:1223 established using socket 0x29a5b0
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico.c:479:
Accepted [socket 1/0x29a5b0]
INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:225:
starting server read loop
```

Code Block 27: RPi 3 B+ Terminal

In our `netcat` terminal we can then input the String we want to send by typing in "hello" and hitting <Enter> again:

```
$ nc 10.0.0.11 5555
hello
```

Code Block 28: PC Terminal

If everything went fine, the network server responds and we receive the echo in our `netcat` terminal.

```
$ nc 10.0.0.11 5555
hello
hello
```

Code Block 29: PC Terminal

TestApp as network server

We can then close the network connection by using <Ctrl + C>. The network server on the RPi will react accordingly

```
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico.c:601:
[socket 1/0x29a5b0] read() unblocked due to connection closed
ERROR: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:235:
socket_read() failed, error -1
```

Code Block 30: RPi 3 B+ Terminal

and will then start storing the received String on our two file system partitions.

```
INFO: /host/sdk/libs/os_filesystem/src/OS_FileSystem.c:125: Maximizing file system
according to size reported by the storage layer (32768 bytes)
INFO: /host/sdk/libs/os_filesystem/src/lib/SpifFs.c:182: Using SPIFFS (phys_addr = 0,
phys_size = 32768, phys_erase_block = 4096, log_block_size = 4096, log_page_size = 256)
INFO: /host/sdk/libs/os_filesystem/src/OS_FileSystem.c:125: Maximizing file system
according to size reported by the storage layer (32768 bytes)
INFO: /host/sdk/libs/os_filesystem/src/lib/SpifFs.c:182: Using SPIFFS (phys_addr = 0,
phys_size = 32768, phys_erase_block = 4096, log_block_size = 4096, log_page_size = 256)
```

Code Block 31: RPi 3 B+ Terminal

After having completed the file system operations, the server will also close the connection.

```
INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:258:
Demo completed successfully.
INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:264:
connection closed by server
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico.c:339:
[socket 1/0x29a5b0] close() handle
```

Code Block 32: RPi 3 B+ Terminal

Notice: During the network and storage operations, at some points the following errors might occur, which can just be ignored for now:

```
system_timer_set_timeout@system_timer.c:66 Timeout in the past
system_timer_set_timeout@system_timer.c:82 Timeout missed
```

Code Block 33: RPi 3 B+ Terminal

4 TestApp as network client

After having seen the server mode, we now want to turn the tables and let the RPi work as network client. In contrast, we want the PC to act as a webserver that will hand out a small webpage. We therefore will

- utilize the Linux tool `netcat` again in order to act as a small webserver
- receive the webpage within our `TestApp`
- store the content of the received webpage in a file on the SPI NOR Flash with the help of our existing file system setup

4.1 What to do

The general setup, the wiring and the IP address configuration between the PC and the RPi will stay the same. We can therefore build upon the network server setup created before but again have to adapt some parts to reflect the new network client setup. In order to solve this self-study, you can follow the guide provided below, which highlights the demo parts that require adaptations and offers additional hints:

- `CMakeLists.txt` remains basically the same, but within the declaration of the `NetworkStack` component we have to switch from server to client
- Within `system_config.h` we have to
 - reduce the maximum number of sockets to 1
 - add a new definition (`CFG_TEST_HTTP_SERVER`) that provides the IP address of our webserver (which in our case is 10.0.0.10, the IP address of the PC's network interface) and can be read from within `TestApp.c`
- Within `TestApp.c` we have to replace the existing network server logic with a respective network client logic (see further hints provided below). In addition, we want to
 - provide the existing array `fileData` with a new array size of 250, as this is the length of the data we will receive from the webserver (basically the content of the HTML page)
 - change the size of `fileSize` to the size of the file content

```
...
static char fileData[250];
...
static const off_t fileSize = sizeof(fileData);
```

Code Block 34: TestApp.c

- We have to provide a small webserver (`netcat` based) which runs on the PC (see section "The netcat webserver" for more information)

4.2 Hints

Regarding the required modifications within `TestApp.c` for being able to work as a network client, please have a look at the requirements documented in the TRENTOS-M Handbook

- chapter "TRENTOS-M Network API" and
- chapter "TRENTOS-M Demo TLS API" (and the respective demo C code)

as well as the RPi 3 B+ IoT Demo (TRENTOS-M Handbook, chapter "TRENTOS-M Demo IoT App for RPi3B+").

For a basic overview, please consult the sections "Initialization of the Client-Side RPC Buffer Pointers" and "TCP Example" of the TRENTOS-M Handbook, chapter "TRENTOS-M Network API". Afterwards, the Demo TLS API shows how to use the TRENTOS-M TLS API in order to retrieve a HTTPS web page. In our case, we don't make use of TLS (and HTTPS), but the basic networking aspects still remain the same. Within the `demo_OS_Tls.c` file, have a look at the functions

- `run()`
- `demoAsLibrary()`
- `readAndPrintWebPage()`

These functions will highlight the fundamental networking workflow. For our setup, you can leave out the crypto and TLS related configuration and setup, and just concentrate on the use of the raw network functionality and its corresponding functions like

- `OS_NetworkSocket_create()`
- `OS_NetworkSocket_write()`
- `OS_NetworkSocket_read()`
- `OS_NetworkSocket_close()`

A similar workflow and additional information can be found in the RPi 3 B+ IoT Demo, within the file `glue_tls_mqtt.c`.

Don't forget: after having received the data from the server, we want to store the received content on the SPI NOR Flash by using the already existing file system and storage functionality!

4.3 The netcat webserver

In order to provide a small sample website, we create a new file `index.html` and fill it with the content provided below:

TestApp as network client

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <h1>Level 1 header</h1>
    <h2>Subheading</h2>
    <p>Normal text here</p>
  </body>
</html>
```

Code Block 35: RPi 3 B+ Terminal

4.4 Run the demo

As already stated above, for the demo we want to provide a small webserver that distributes the HTML file `index.html`. Again, we use `netcat` to provide the respective server facilities. We will host the server at the PC (keeping the 10.0.0.10 IP address), but this time utilize port 8888. As we want to have `netcat` serve the page (`index.html`) indefinitely, we wrap the command in an infinite loop. This will allow `netcat` to continue to receive connections after the first connection closes. We nevertheless can stop the loop at any time by using `<Ctrl + C>`. For now, we start `netcat` and provide it with the path to the respective HTML file.

```
$ while true; do printf 'HTTP/1.0 200 OK\n\n%s' "$(cat index.html)" | netcat -l 8888;
done
```

Code Block 36: RPi 3 B+ Terminal

Now, the webserver runs and we can start the demo on the RPi in the same way as in the assignments before. A successful run should provide you with an output similar like the one below:

TestApp as network client

```
U-Boot 2020.07-rc3-00002-g4b6f96a0a6 (May 26 2020 - 18:38:13 +0200)

DRAM:  948 MiB
RPI 3 Model B+ (0xa020d3)
MMC:   mmc@7e202000: 0, sdhci@7e300000: 1
Loading Environment from FAT... OK
In:     serial
Out:    vidconsole
Err:    vidconsole
Net:    No ethernet found.
starting USB...
Bus usb@7e980000: USB DWC2
scanning bus usb@7e980000 for devices... 4 USB Device(s) found
    scanning usb for storage devices... 0 Storage Device(s) found
Hit any key to stop autoboot:  0
8336100 bytes read in 348 ms (22.8 MiB/s)
## Starting application at 0x00b27000 ...

ELF-loader started on CPU: ARM Ltd. Cortex-A53 r0p4
    paddr=[b27000..12fb4c7]
No DTB passed in from boot loader.
Looking for DTB in CPIO archive...found at c34294.
Loaded DTB from c34294.
    paddr=[3c000..3ffff]
ELF-loading image 'kernel'
    paddr=[0..3bfff]
    vaddr=[e0000000..e003bfff]
    virt_entry=e0000000
ELF-loading image 'capdl-loader'
    paddr=[40000..728fff]
    vaddr=[10000..6f8fff]
    virt_entry=1860c
Enabling MMU and paging
Jumping to kernel-image entry point...

Bootstrapping kernel
Booting all finished, dropped to user space
main@main.c:2114 Starting CapDL Loader...
main@main.c:2116 CapDL Loader done, suspending...
spt_handle_irq@spt.c:172 handle irq called when no interrupt pending

INFO: /host/sdk/demos/demo_training_course/src/components/Ticker/Ticker.c:15: ticker
running
INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:169:
Starting t    INFO:
/host/sdk/demos/demo_training_course/src/components/NetworkStack/NetworkStack.c:51:
```

TestApp as network client

```
[NwStack 'nwStack'] startin
DEBUG uspi: - Initializing USPi library 2.00WARNING:
/host/sdk/components/NIC_RPi/environment.c:104: Not impleme INFO:
/host/sdk/components/RPi_SPI_Flash/RPi_SPI_Flash.c:131: BCM2837_SPI_Flash init
INFO: /host/sdk/componeest_app_server...
g
nted!
nts/RPi_SPI_Flash/RPi_SPI_Flash.c:196: BCM2837_SPI_Flash done
INFO: /host/sdk/components/StorageServer/server/src/storageServer_rpc.c:504: storage
medium size: 8388608 bytes
INFO: /host/sdk/components/StorageServer/server/src/storageServer_rpc.c:522: client 1:
offset=0, size=32768
INFO: /host/sdk/components/StorageServer/server/src/storageServer_rpc.c:522: client 2:
offset=32768, size=32768

NOTICE usbdev0-1: - Device ven424-2514, dev9-0-2 found
NOTICE usbdev0-1: - Interface int9-0-1 found
WARNING usbdev0-1: - Function is not supported
NOTICE usbdev0-1: - Interface int9-0-2 found
NOTICE usbdev: - Using device/interface int9-0-2
NOTICE usbdev0-1: - Device ven424-2514, dev9-0-2 found
NOTICE usbdev0-1: - Interface int9-0-1 found
WARNING usbdev0-1: - Function is not supported
NOTICE usbdev0-1: - Interface int9-0-2 found
NOTICE usbdev: - Using device/interface int9-0-2
NOTICE usbdev0-1: - Device ven424-7800 found
NOTICE usbdev: - Using device/interface ven424-7800
DEBUG lan7800: - MAC address is B8:27:EB:52:4A:44
DEBUG usbhub: - Port 1: Device configured
DEBUG usbhub: - Port 1: Device configured
DEBUG dwroot: - Device configured
DEBUG uspi: - USPi library successfully initialized INFO:
/host/sdk/components/NIC_RPi/ethernetDriver.c:44: [EthDrv 'nic'] starting
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico_nic.c:214:
MAC: b8:27:eb:52:4a:44
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico_nic.c:225:
PicoTCP Device created: trentos_nic_driver
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_core.c:404:
signal network stack init done
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_config.c:232:
notify_app_init_done
WARNING: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico_nic.c:124:
Falling back to legacy interface.
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico.c:310:
[socket 0/0x19f358] created new socket
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico.c:217:
```

TestApp as network client

```
[socket 0x19f358] incoming connection established
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico.c:377:
[socket 0/0x19f358] connection established to 10.0.0.10:8888
INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:200:
Send request to host...
INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:212:
Writing request to socket 0 for GET / HTTP/1.0
H
INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:238:
read response...
INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:284:
chunk read, length 250, handle 0
```

Code Block 37: RPi 3 B+ Terminal

We can see that the network client running on the RPi successfully connected to the host PC, has sent a request and already received the correct response. The received content has a total length of 250 characters. The netcat server also shows the corresponding connection.

```
GET / HTTP/1.0
Host: 10.0.0.10
Connection: close
```

Code Block 38: RPi 3 B+ Terminal

As the communication went just fine, in a next step we will now close the netcat server by using >Ctrl + C>.

```
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico.c:601:
[socket 0/0x199ae8] read() unblocked due to connection closed
INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:278:
socket_read() reported connection closed for handle 0
```

Code Block 39: RPi 3 B+ Terminal

The network client acknowledges accordingly. In a last step, the received content is then written to a new file on the two partitions offered by the SPI NOR Flash (like we did in all of the assignments before).

TestApp as network client

```
INFO: /host/sdk/libs/os_filesystem/src/OS_FileSystem.c:125: Maximizing file system
according to size reported by the storage layer (32768 bytes)
INFO: /host/sdk/libs/os_filesystem/src/lib/SpifFs.c:182: Using SPIFFS (phys_addr = 0,
phys_size = 32768, phys_erase_block = 4096, log_block_size = 4096, log_page_size = 256)
INFO: /host/sdk/libs/os_filesystem/src/OS_FileSystem.c:125: Maximizing file system
according to size reported by the storage layer (32768 bytes)
INFO: /host/sdk/libs/os_filesystem/src/lib/SpifFs.c:182: Using SPIFFS (phys_addr = 0,
phys_size = 32768, phys_erase_block = 4096, log_block_size = 4096, log_page_size = 256)
INFO: /host/sdk/demos/demo_training_course/src/components/TestApp/src/TestApp.c:310:
Demo completed successfully.
INFO: /host/sdk/libs/os_network_stack/network_core/src/network_stack_pico.c:339:
[socket 0/0x199ae8] close() handle
```

Code Block 40: RPi 3 B+ Terminal



HENSOLDT Cyber GmbH

Willy-Messerschmitt-Straße 3
82024 Taufkirchen,
Deutschland

www.hensoldt-cyber.com